Instrumentation

Introduction à Python, NumPy et Matplotlib

Marc Nicollerat Marion Badi

Chapitre 2 — Python comme outil d'analyse

Instrumentation 2026-2027

2.1 Mais pourquoiiiiii !

Il y a plusieurs possibilités pour traiter des données :

- Excel, très intuitif, peut devenir compliqué avec les formules complexes
- LABVIEW, graphique, support d'interfaces en tout genre, plutôt orienté acquisition
- MATLAB, un standard pour les riches ! Plein de librairies.
- Python, générique, gratuit, plein d'interface, polyvalent…

Objectifs et organisation du TP

À l'issue de ce TP, l'étudiant·e est capable de :

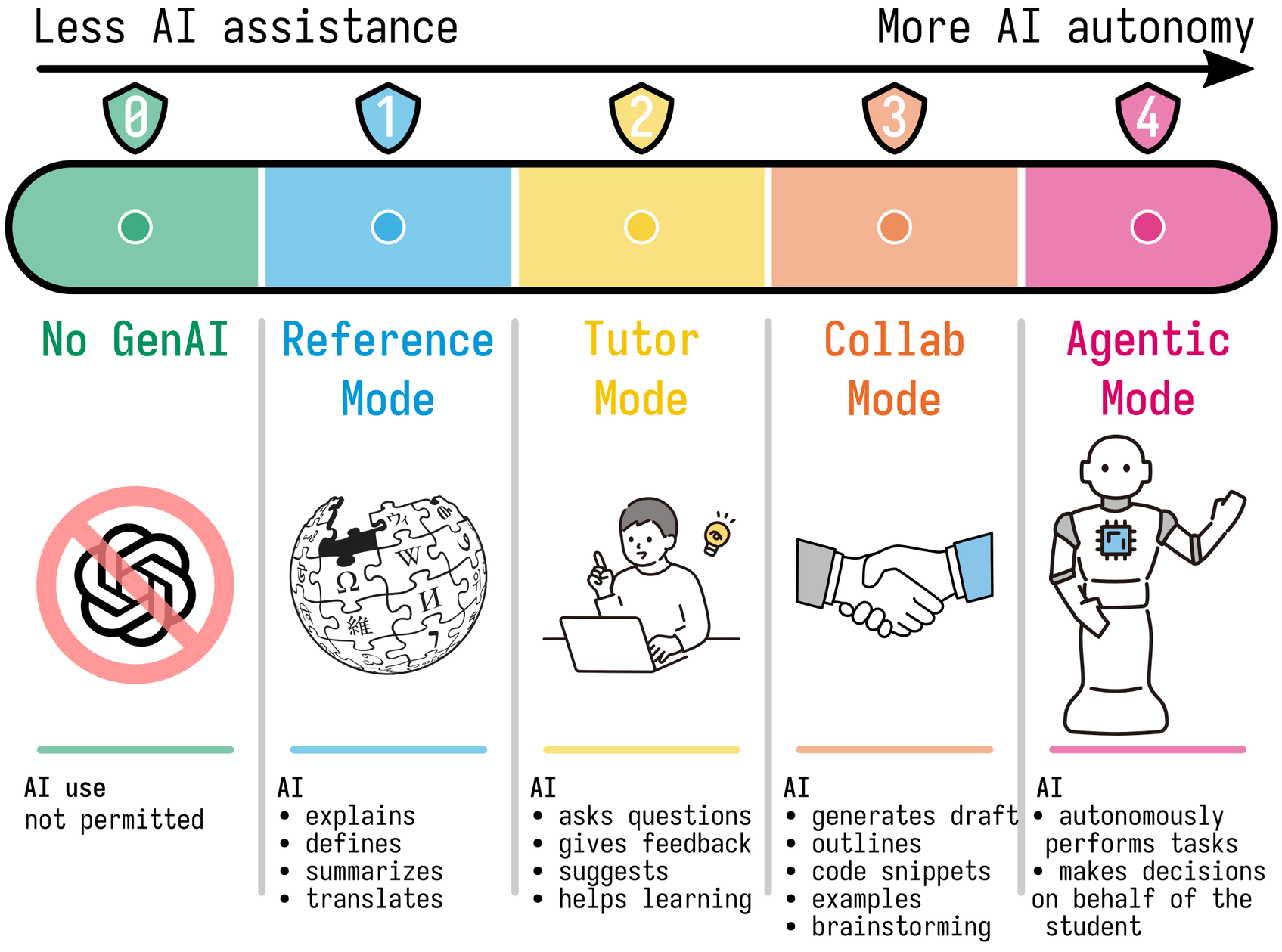
1. **Configurer** un environnement Python reproductible avec `uv` et importer les bibliothèques scientifiques nécessaires (`numpy`, `matplotlib`, `pandas`).
2. **Importer** un fichier de mesures au format CSV sous forme de tableau `numpy` et/ou de `DataFrame pandas`.
3. **Tracer** un signal avec `matplotlib` en respectant les règles de l'art (titre, labels, unités, échelles, légende).
4. **Exploiter un outil de traitement de données** (`pandas`) pour explorer, filtrer et manipuler un jeu de mesures tabulaire.
5. **Manipuler des vecteurs** avec `numpy` pour effectuer des opérations vectorisées : inverser une caractéristique de capteur (affine ou non linéaire), calculer des statistiques descriptives, ou d'autres traitements.

Support de cours :

- Ce document
- Données pour des tests disponibles sur cyberlearn

Utilisation de l'IA

L'objectif du cours est de comprendre et de maîtriser les scripts !



Niveau d'utilisation

- Pour apprendre, l'usage de l'AI doit être minimum. L'AI est utile pour se documenter
- La réalisation de TPs peut se faire avec plus de support
- **Les tests se font sans l'AI.**

Instrumentation 2026-2027

2.2 Configuration de l'environnement avec uv

uv est un gestionnaire de paquets et d'environnements Python moderne qui remplace avantageusement la combinaison `pip` + `venv` + `virtualenv` : il est très rapide, gère un fichier de verrouillage (`uv.lock`) pour garantir la reproductibilité, et sait aussi installer des versions de Python.

2.2.1 Installation de uv

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Vérifier l'installation :

```
uv --version
```

2.2.2 Installer et sélectionner une version précise de Python

uv peut télécharger et installer une version précise de Python. Par exemple, pour utiliser Python 3.12 :

```
1 uv python install 3.12
```

Pour utiliser cette version, exécuter :

```
1 uv python pin 3.12
```

Pour lancer un interpréteur python :

```
1 uv run python
```

Pour quitter l'interpréteur

```
1 >>>quit()
```

2.2.3 Créer un projet et configurer

Pour créer un projet, utiliser ces commandes.

```
uv init tp-capteurs
cd tp-capteurs
```

Cette commande crée un `pyproject.toml` minimal qui décrit le projet (nom, version, dépendances). C'est ce fichier et le `uv.lock` associé qui garantit que n'importe qui peut recréer exactement le même environnement.

2.2.4 Ajouter les bibliothèques nécessaires

Dans le répertoire du projet, il est possible d'ajouter des bibliothèques :

```
1  uv add numpy matplotlib pandas
```

`uv` crée automatiquement un environnement virtuel isolé (`.venv/`) et y installe les paquets demandés, sans que vous ayez à l'activer manuellement.

2.2.5 Exécuter un script

La commande de base est

```
1 python script.py`
```

mais ceci ne fonctionne qu'avec la version de python disponible **dans le path** de votre système, s'il y en a un ! Vous pouvez **activer** l'environnement du projet avec la commande suivante :

```
1 .venv\Scripts\Activate.ps1
```

Avec uv, on peut exécuter un script de cette façon, qui active le bon environnement selon la définition de votre projet :

```
1 uv run python mon_script.py
```

ou

```
1 uv run mon_script.py
```

`uv run` s'assure que l'environnement est synchronisé avec `pyproject.toml` **avant** chaque exécution : il n'y a jamais de décalage entre les dépendances déclarées et celles réellement installées.

2.2.6 Importer les bibliothèques en Python

Convention quasi universelle dans la communauté scientifique Python :

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Module	Rôle	Alias conventionnel
`numpy`	calcul numérique, tableaux `ndarray`	`np`
`matplotlib.pyplot`	tracé de graphiques	`plt`
`pandas`	manipulation de données tabulaires	`pd`

2.2.7 Exercice pratique

⚠ L'austère ligne de commande

Ouvrez un terminal dans un répertoire de votre choix et testez la ligne de commande.

```
uv run --with matplotlib --with numpy python
```

puis dans la console, par exemple :

```
import numpy as np
import matplotlib.pyplot as plt

tableau = np.array([1, 2, 3, 4, 5])
print(tableau)
tableau*2

plt.plot(tableau, marker="o")
plt.xlabel("Indice")
plt.ylabel("Valeur")
plt.show()
```

un peu de confort en vue

Il y a des environnements pour aider à développer du code, avec éditeur, debuggeur

- Jupyter Notebook
- Anaconda, Eric,…
- VSCode

2.2.8 Exercice

⚠ La sordide histoire du répertoire de travail

Essayez de jouer ce code. Créez un fichier avec le code et lancez-le depuis la ligne de commande.

```python
from pathlib import Path


def main():
    print("Hello from demonstration!")
    print(Path.cwd())


if __name__ == "__main__":
    main()
```

Instrumentation 2026-2027

2.3 Les notebooks Jupyter

Un notebook Jupyter est un document interactif, généralement enregistré avec l'extension `.ipynb`. Il permet de mélanger du texte formaté, du code Python, des résultats, des tableaux et des graphiques dans un même document. Les cellules de code sont exécutées individuellement, ce qui est pratique pour analyser des données et tester progressivement un script.

2.3.1 Installer Jupyter

Dans un projet utilisant uv, on peut ajouter Jupyter comme dépendance de développement :

```
uv add --dev jupyter
```

Les bibliothèques utilisées dans les notebooks doivent également être installées dans le projet, par exemple :

```
uv add numpy matplotlib pandas
```

hors uv

Une installation globale est aussi possible, mais elle est moins reproductible :

```
python -m pip install jupyterlab numpy matplotlib pandas
```

2.3.2 Créer et ouvrir un notebook

Depuis le répertoire racine du projet, lancer JupyterLab avec :

```
uv run jupyter lab
```

Une page s'ouvre alors dans le navigateur. Dans le navigateur de fichiers, cliquer sur **File > New > Notebook**, choisir le noyau Python, puis donner un nom au fichier, par exemple `exploration.ipynb`.

Une alternative plus légère est l'interface classique :

```
uv run jupyter notebook
```

Le notebook peut aussi être ouvert directement depuis Positron ou Visual Studio Code, à condition qu'une extension Jupyter soit installée.

2.3.3 Utiliser les cellules

Une cellule peut contenir du code Python ou du texte Markdown. Pour exécuter la cellule sélectionnée, utiliser le bouton **Run** ou le raccourci `Shift+Enter`. Le résultat apparaît immédiatement sous la cellule.

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.show()
```

Les cellules Markdown servent à documenter l'analyse : titres avec `#`, listes, liens, formules LaTeX et explications peuvent être ajoutés entre les cellules de code.

Cellule markdown

```
# Titre

## sous-titre

Relation de base :

$$
e=m c^2
$$
```

2.3.4 Comprendre l'ordre d'exécution

Lorsqu'on joue avec un notebook, on n'exécute pas nécessaire les cellules dans l'ordre où elles apparaissent. Une variable créée dans une cellule reste disponible dans le noyau jusqu'à son redémarrage :

```
1 mesures = np.array([1.2, 1.5, 1.4, 1.8])
2 mesures.mean()
```

Pour obtenir un résultat reproductible, il faut forcer l'exécution de toutes les cellules depuis le début, par exemple avec **Kernel > Restart Kernel and Run All Cells**.

Il est recommandé de placer les imports et la configuration au début du notebook, et d'éviter de dépendre d'une exécution manuelle dans un ordre particulier.

2.3.5 Lire et exporter un notebook

Un notebook se sauvegarde avec `Ctrl+S`. Le fichier `.ipynb` contient le code, le texte et, éventuellement, les résultats enregistrés. Il peut être partagé avec d'autres personnes, mais celles-ci doivent disposer des mêmes dépendances pour réexécuter le code.

Pour produire un document HTML ou PDF depuis le terminal :

```
uv run jupyter nbconvert --to html exploration.ipynb
uv run jupyter nbconvert --to pdf exploration.ipynb
```

2.3.6 Installer les outils nécessaires pour exporter en PDF

Pour exporter un notebook `.ipynb` en PDF, il faut installer `nbconvert`, Pandoc et une distribution LaTeX. Sous Windows, **MiKTeX** est une solution simple.

1. Dans le terminal du projet, installer `nbconvert` :

```
uv add nbconvert
```

2. Installer Pandoc, puis vérifier l'installation :

```
pandoc --version
```

3. Installer MiKTeX. Pendant l'installation, choisir l'option permettant d'installer automatiquement les paquets LaTeX manquants. Vérifier ensuite que LaTeX est accessible :

```
pdflatex --version
```

4. Relancer l'exportation :

```
uv run jupyter nbconvert --to pdf exploration.ipynb
```

Instrumentation 2026-2027

2.4 Python et Jupyter dans VS Code

2.4.1 Installer les extensions

Dans VS Code, ouvrir le dossier du projet puis installer les extensions **Python** et **Jupyter** de Microsoft. Les fichiers `.py` et `.ipynb` peuvent alors être exécutés directement dans l'éditeur.

> ⚠ **Premier pas avec VSCode**
>
> - Choisissez un répertoire de travail avec **File->Open Folder**.
> - Créez un script `test.py` qui importe numpy, génère un vecteur numpy avec les nombres de 1 à 10, calcule et affiche la somme (fontion np.sum)

2.4.2 Créer l'environnement du projet

Lors de la première exécution d'un script, VSCode demande quel interpéteur python choisir. Si un environnement a été défini, il devrait l'utiliser par défaut.

VSCode permet d'utiliser un terminal. (**Terminal > New Terminal**) L'environnement peut être définit et modifié :

```
1 uv python install 3.12
2 uv python pin 3.12
3 uv add numpy matplotlib pandas ipykernel
```

`uv` crée et utilise l'environnement `.venv`, dont les dépendances sont déclarées dans `pyproject.toml`.

2.4.3 Sélectionner Kernel Python ou Jupyter

- `Ctrl+Shift+P` → **Python: Select Interpreter** → sélectionner `.venv\Scripts\python.exe` sous Windows (ou `.venv/bin/python` sous macOS/Linux) ;
- créer ou ouvrir un fichier `.ipynb` ;
- cliquer sur **Select Kernel** en haut à droite et sélectionner le même environnement `.venv` ;
- exécuter une cellule avec `Shift+Enter`.

L'interpréteur est utilisé par les scripts `.py` ; le kernel est utilisé par les cellules Jupyter.

2.4.4 Utiliser la console pour trouver les erreurs

Lancer un script dans le terminal intégré :

```
1 uv run python analyse.py
```

La console affiche le *traceback*. Le lire de bas en haut : la dernière ligne indique le type d'erreur et son message, tandis que les lignes précédentes indiquent le fichier et le numéro de ligne concernés. Dans un notebook, l'erreur apparaît sous la cellule ; le débogueur de VS Code permet de placer un point d'arrêt et d'inspecter les variables.

> **Tip**
>
> Si un module est introuvable, vérifier d'abord l'interpréteur ou le noyau sélectionné, puis installer la dépendance avec `uv add nom-du-paquet`.

Instrumentation 2026-2027

2.5 Librairies python pour le traitement de données

Python n'est pas seulement un langage de programmation, il propose un catalogue de librairies assez large pour différentes applications. Nous allons en particulier utiliser `numpy` et `matplotlib` pour traiter des données et les afficher sous forme graphique. `pandas` est aussi une librairie intéressante pour la gestion de base de données.

2.5.1 Trouver des fichiers dans les répertoires

L'organisation d'un projet se fait assez rapidement avec différents répertoires pour classer les données, organiser les scripts, etc.

Prenons une organisation de projet typique :

```
mon_projet/              <- répertoire ouvert dans Positron (racine du projet)
├── script/
│   └── analyse.py        <- le script que vous éditez et exécutez
└── data/
    └── mesures.csv        <- le fichier que vous voulez lire
```

La solution pour les scripts

```python
from pathlib import Path

# dossier contenant le script en cours d'exécution (ex. .../mon_projet/script/)
script_dir = Path(__file__).resolve().parent
print("Répertoire du script : ", scrip_dir)

# chemin vers data/, exprimé PAR RAPPORT AU SCRIPT — exactement "../data/..."
chemin_fichier = script_dir / "../data/mesures.csv"

df = pd.read_csv(chemin_fichier)
```

Setup expérimental

Récupérez le projet du cyberlearn avec les différents fichiers en respectant les répertoires.

2.5.2 Problème dans jupyter notebook

`__file__` n'est défini que dans un fichier `.py` exécuté normalement. Il n'existe pas dans une console Python interactive, et se comporte différemment dans certains notebooks Jupyter.

Pour debugger un problème, on peut anylser dans l'onglet "Terminal" la ligne de commande utilisée :

```
PS C:\Users\marc.nicoller\OneDrive\OneDrive - HESSO\Enseignement\Instrumentation> & C:\Users\marc.
nicoller\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/
marc.nicoller/OneDrive/OneDrive - HESSO/Enseignement/Instrumentation/
Cours_2026/cours-python-numpy/scripts/generate_ntc_bouilloire.py"
```

Ceci donne le répertoire de travail et permet d'ajuster le path sur le fichier.

2.5.3 Lire des données avec NumPy

Pour travailler des données avec NumPy, il faut d'abord les charger d'un fichier de mesure, par exemple un fichier CSV. Numpy ne traite que des nombres ! Il faut donc ignorer les en-têtes qui ont les noms de données

```
1     # code qui lit
2     donnees = np.loadtxt("cours-python-numpy/data/fichier.csv", delimiter=",", skiprows=1)
3
4     print("Format : ",donnees.shape)
5     print("Données :",  donnees)
```

⚠ Exercice

Préparez un répertoire de travail avec un sous-répertoire data et un sous-répertoire scripts. Récupérez de cyberlearn les fichiers csv et copiez-les dans le répertoire `/data`.

- Ecrivez un script que vous sauvez dans le répertoire `/script` qui lit le fichier "etalonnage_capteur.csv" dans le répertoire data.
- Faites en sorte que votre script trouve par lui-même le chemin vers les données en se basant sur le répertoire actuel du script.
- Le répertoire du script peut être connu par la fonction pathlib.Path(**file**).resolve().parent

⚠ Où suis-je ?

- Vous pouvez utiliser `pathlib.Path.cwd()` pour savoir dans quel répertoire vous êtes
- Si vous inspectez le terminal, vous avez le path courant utilisé par Python.
-

2.5.4 Solution

▶ Code

> **Alternative**
>
> `np.genfromtxt(..., delimiter=",", names=True)` est une alternative à `np.loadtxt`, plus tolérante (valeurs manquantes, types mixtes) et permettant d'accéder aux colonnes par leur nom d'en-tête (`donnees["X_ref_bar"]`).

2.5.5 Lire des données avec pandas

Pandas est plus orienté vers les bases de données. Cette librairie est adaptée dès que l'on veut manipuler les colonnes par leur **nom**, mélanger types numériques et texte, ou faire des opérations de type tableur (filtrage, agrégation…).

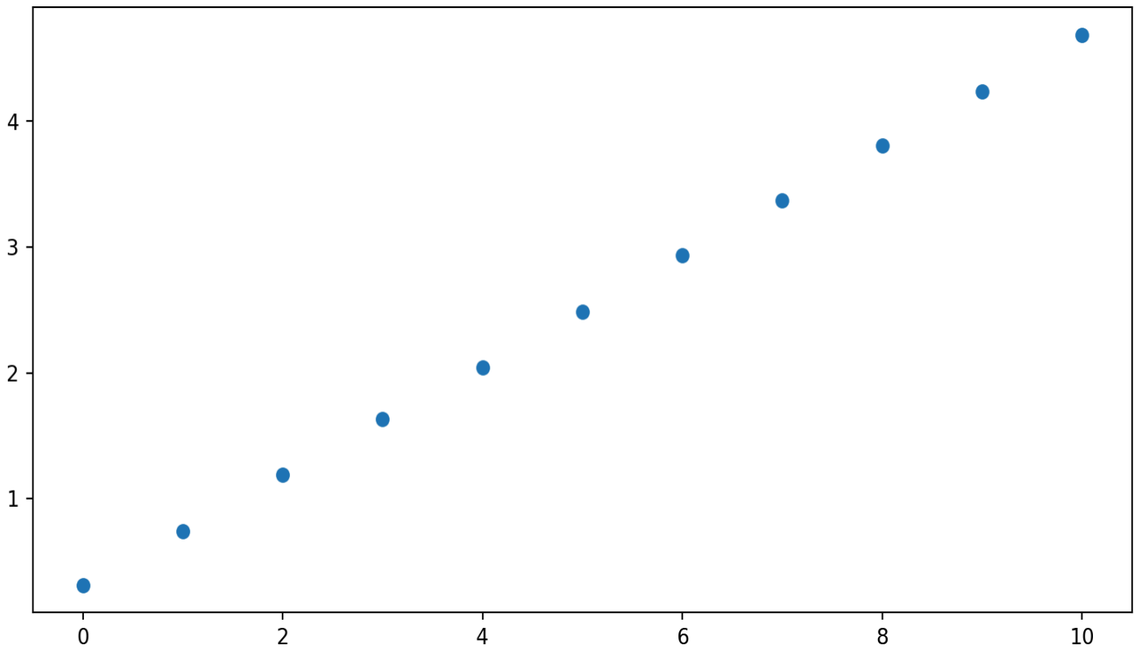
```python
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv(script_dir / "data/etalonnage_capteur.cs
print(df.head())
print(df.columns.tolist())

X_ref = df["X_ref_bar"].to_numpy()   # -> retour à un tab
V_mes = df["V_mesure_V"].to_numpy()

plt.plot(X_ref, V_mes, 'o')
```

```
   X_ref_bar  V_mesure_V
0        0.0      0.3144
1        1.0      0.7407
2        2.0      1.1920
3        3.0      1.6305
4        4.0      2.0444
['X_ref_bar', 'V_mesure_V']
```



2.5.6 Manipuler des vecteurs avec NumPy

`Numpy` utilise un format de tableau (`ndarray`) qui permet d'effectuer des opérations mathématique sur l'ensemble du tableau. Voici comment les **créer**, les **transformer** (notamment de façon affine), et en extraire des **indicateurs statistiques** simples.

2.5.7 Créer un vecteur (tableau 1D)

Voici quelques façons de créer des tableaux numpy.

```python
import numpy as np

# à partir d'une liste
X = np.array([0.0, 1.0, 2.0, 3.0, 4.0])

# tableau de zéros / de uns
Z = np.zeros(5)
U = np.ones(5)

# tableau de CONSTANTE (valeur au choix)
C = np.full(5, 3.3)          # 5 valeurs valant 3.3

# suite arithmétique par pas (borne sup exclue)
A = np.arange(0, 10, 2)      # [0, 2, 4, 6, 8]

# tableau LINÉAIRE : n points régulièrement espacés entre deux bornes
L = np.linspace(0, 10, 5)    # [0., 2.5, 5., 7.5, 10.] (bornes incluses)
```

`np.arange` vs `np.linspace`

- `np.arange(debut, fin, pas)` : on fixe le **pas**, le nombre de points en résulte (et la borne finale est **exclue**).
- `np.linspace(debut, fin, n)` : on fixe le **nombre de points**, le pas en résulte (et la borne finale est **incluse** par défaut).

2.5.8 Opérations vectorielles : la transformation affine

L'intérêt principal de `numpy` est de pouvoir appliquer une opération à **tout un tableau d'un coup** (opération dite *vectorisée*), sans boucle `for` explicite :

```python
import numpy as np
a, b = 0.437, 0.314          # gain et offset (ex. issus d'un étalonnage)

X = np.array([0.0, 1.0, 2.0, 3.0, 4.0])

Y = a * X + b                 # transformation affine appliquée à TOUT le vecteur
# Y = array([0.314, 0.751, 1.188, 1.625, 2.062])
```

C'est exactement cette mécanique qui est utilisée pour **corriger** un signal mesuré (transformation affine inverse) :

```python
V_mesure = np.array([0.31, 0.75, 1.19, 1.63])   # tension mesurée (V)

X_estime = (V_mesure - b) / a                    # correction inverse, vectorisée
```

> **Pourquoi pas de boucle `for` ?**
>
> On pourrait écrire `[( v - b) / a for v in V_mesure]`, mais la version `numpy` vectorisée est à la fois **plus courte**, **plus lisible**, et **beaucoup plus rapide** (le calcul est délégué à du code compilé, pas interprété élément par élément). La ligne ci-dessus ne retourne pas un tableau numpy, c'est une *liste* d'éléments, on ne peut plus faire d'opérations dessus.

2.5.9 Break pratique

Exercice

Créez un tableau numpy qui représente des angles entre 0 et 2π, puis utisez la fonction `np.sin` pour calculer le sinus de ces angles.

2.5.10 Statistiques descriptives : max, min, moyenne, variance

Numpy permet de calculer des statistique sur les données de ses tableaux :

```python
script_dir = Path(__file__).resolve().parent
X_ref, V_etal = np.loadtxt(
    script_dir / "data/etalonnage_capteur.csv", delimiter=",", skiprows=1, unpack=True
)

v_max = np.max(V_etal)        # équivalent : V_etal.max()
v_min = np.min(V_etal)        # équivalent : V_etal.min()
v_moy = np.mean(V_etal)       # moyenne
v_var = np.var(V_etal)        # variance
v_std = np.std(V_etal)        # écart-type = racine carrée de la variance

print(f"min={v_min:.3f} V, max={v_max:.3f} V")
print(f"moyenne={v_moy:.3f} V, variance={v_var:.5f} V², écart-type={v_std:.3f} V")
```

```
min=0.314 V, max=4.689 V
moyenne=2.495 V, variance=1.91032 V², écart-type=1.382 V
```

2.5.11 Quelques fonctions utiles

Lorsqu'on a un ensemble de mesures, il est intéressant de calculer quelques valeurs statistiques simples :

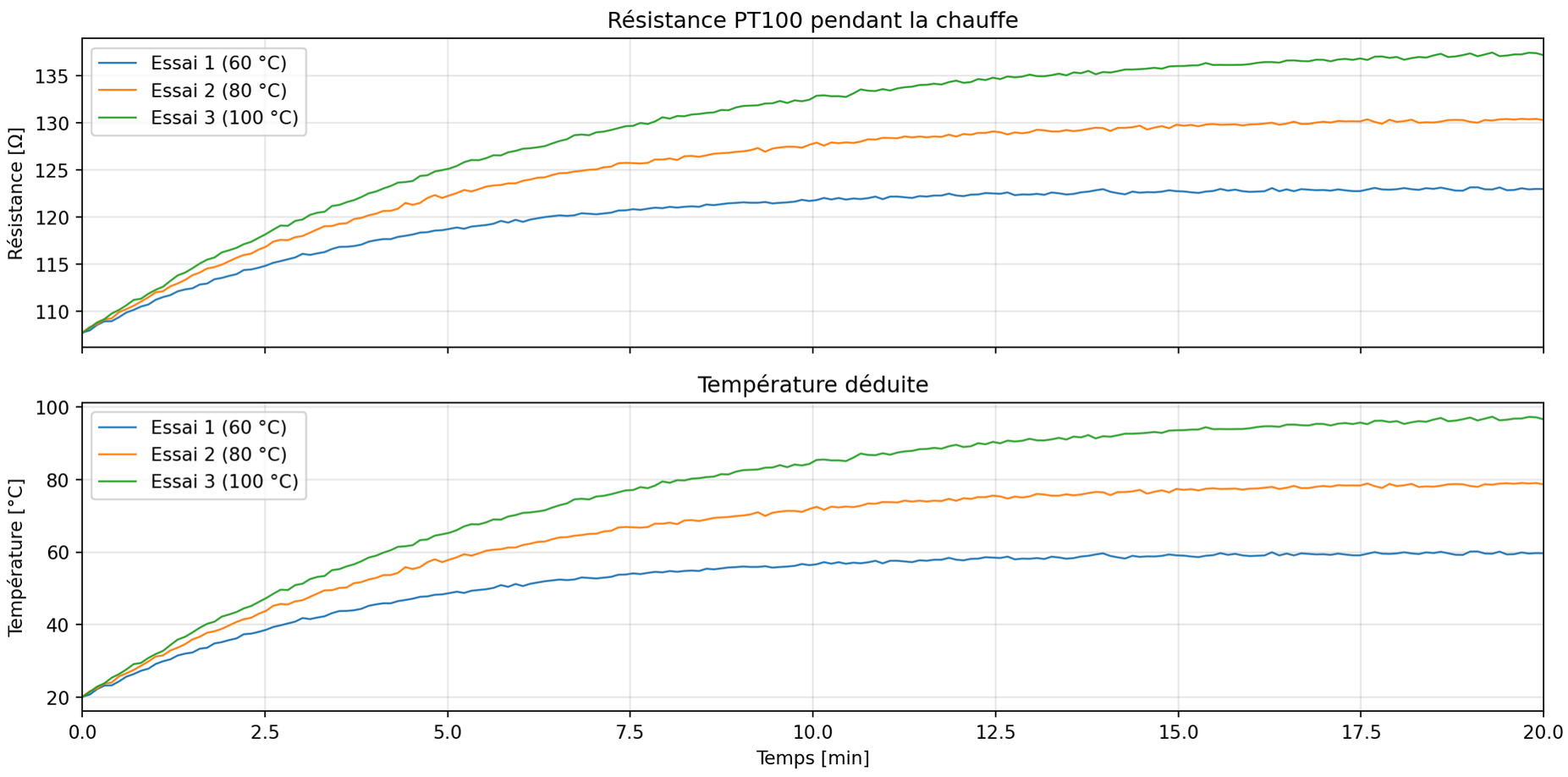
Fonction	Calcule	Même unité que les données ?
`np.max` / `np.min`	valeur extrême du tableau	oui
`np.mean`	moyenne arithmétique	oui
`np.var`	variance (dispersion autour de la moyenne)	**non** (unité au carré)
`np.std`	écart-type = $\sqrt{\text{variance}}$	oui

Instrumentation 2026-2027

2.6 Tracer une courbe avec Matplotlib

Un graphique est un moyen privilégié pour communiquer des résultats. Il doit contenir toutes les informations nécessaires, en respectant un formalisme. Les données doivent être présentées avec pertinence, il faut des échelles, des unités, on peut comparer des résultats, montrer différentes données. Matplotlib permet de faire tout ça à travers un certain nombre de fonctions. Voici un exemple de présentation

▶ Code



2.6.1 Fonctions fréquentes

- `fig, (ax_r, ax_t) = plt.subplots(2, 1, figsize=(12 , 6), sharex=True)` crée la figure, 2 plots en ligne ;
- `ax_r.plot(t, R[i], color=couleurs[i], lw=1, label=label)` trace la courbe sur l'axe r, spcifie une couleur, une épaisseur et un label;
- `ax.set_xlim` / `ax.set_ylim` permet d'imposer les valeurs limites des 2 axes ;
- `ax.set_xlabel` / `ax.set_ylabel` permet de définir l'axe, par ex. `Tension [V]`, `Temps [s]`, `Température [°C]` ;
- `ax_t.grid(True, alpha=0.3)` trace la grille ;
- `ax_t.legend(loc="best")` trace les légendes spécifiées avec spécifiées lors du plot avec le paramètre `label`;
- `plt.tight_layout()` avant `savefig`/`show` évite que les labels ne se superposent.

2.6.2 Enregistrer la figure en image (PNG)

Certaines des figures demandées dans ce TP doivent être **publiées sur le Padlet du cours** : il faut donc savoir les exporter en image, typiquement au format `.png`. Ceci est utile en pratique pour les insérer dans un document.

```
1 fig.savefig(script_dir / "img/signal_brut.png", dpi=150, bbox_inches="tight")
```

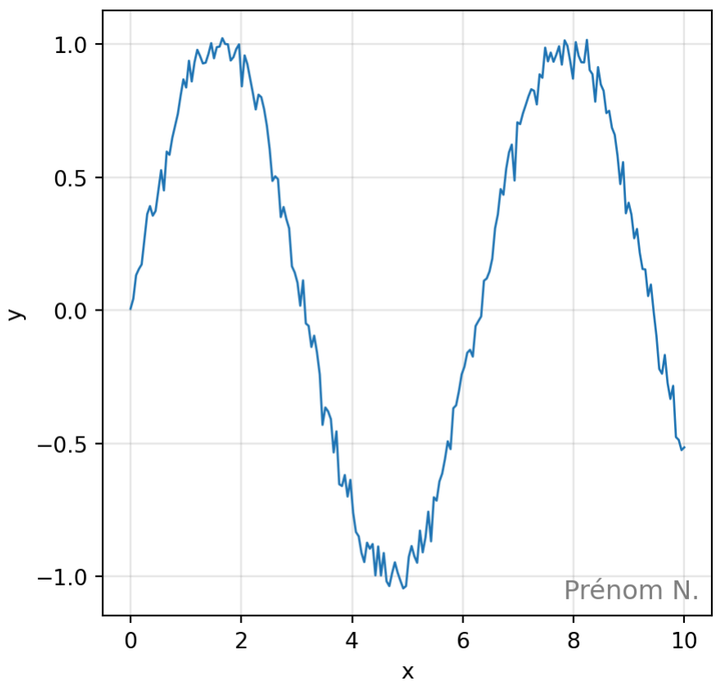
- `dpi=150` (voire 200) donne une image suffisamment nette pour être lue à l'écran comme à l'impression ;
- `bbox_inches="tight"` évite que le titre ou les labels des axes soient rognés sur les bords de l'image ;
- `fig.savefig(...)` doit être appelé **avant** `plt.show()`, qui vide la figure de la mémoire une fois affichée ;

2.6.3 Exemple avec personnalisation

Personnaliser une figure avant export

```python
fig, ax = plt.subplots(figsize=(4
ax.plot(x, y, color="tab:blue", l

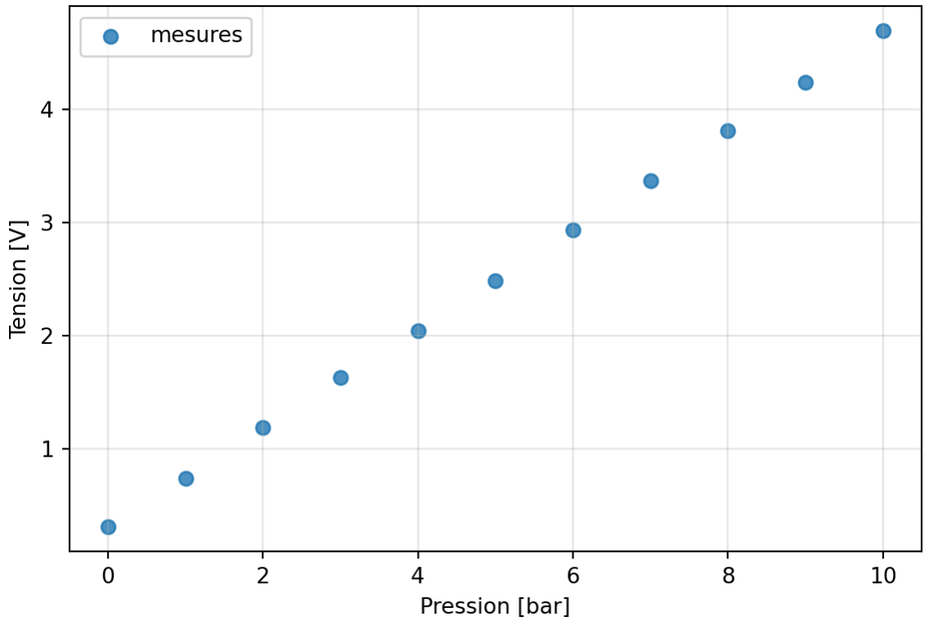
# Nom en texte, coin bas-droit
ax.text(
    0.98, 0.02, "Prénom N.",
    transform=ax.transAxes,
    ha="right", va="bottom",
    fontsize=8, color="gray",
)
```



2.6.4 Tracer un nuage de points

Pour indiquer des mesures à des valeurs particulières, il peut être pertinent de ne marquer que des points.

```
1  fig, ax = plt.subplots(figsize=(7, 4.5))
2  ax.scatter(X_ref, V_etal, color="tab:blue", s=40, alpha=0
3  ax.set_xlabel("Pression [bar]")
4  ax.set_ylabel("Tension [V]")
5  ax.grid(True, alpha=0.3)
6  ax.legend()
```



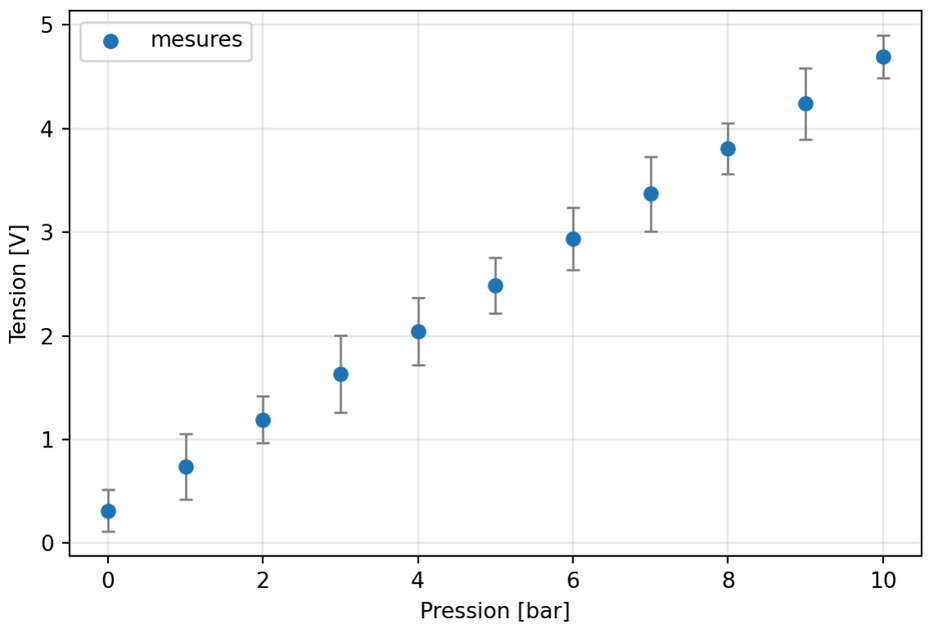
- `s=` contrôle la taille des marqueurs, `alpha=` leur transparence (utile si des points se superposent) ;
- `ax.scatter` accepte les mêmes options de couleur/label que `ax.plot`.

2.6.5 Indiquer une variance (incertitude) sur une trace

Si l'on connaît l'écart-type du bruit de mesure (ex. `sigma = np.std(...)`, ou une valeur constante donnée par le datasheet), on peut l'afficher point par point avec `ax.errorbar`.

```python
# On invente une variance...
yerr = np.random.random(size=len(X_ref))*0.19+0.2

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.scatter(X_ref, V_etal, color="tab:blue", zorder=3, lab
ax.errorbar(
    X_ref, V_etal, yerr=yerr,
    fmt="none",            # pas de marqueur/ligne supple
    ecolor="gray", elinewidth=1, capsize=3,
)
ax.set_xlabel("Pression [bar]")
ax.set_ylabel("Tension [V]")
ax.grid(True, alpha=0.3)
ax.legend()
```



2.6.6 Contrôler les échelles : divisions, sous-divisions, labels

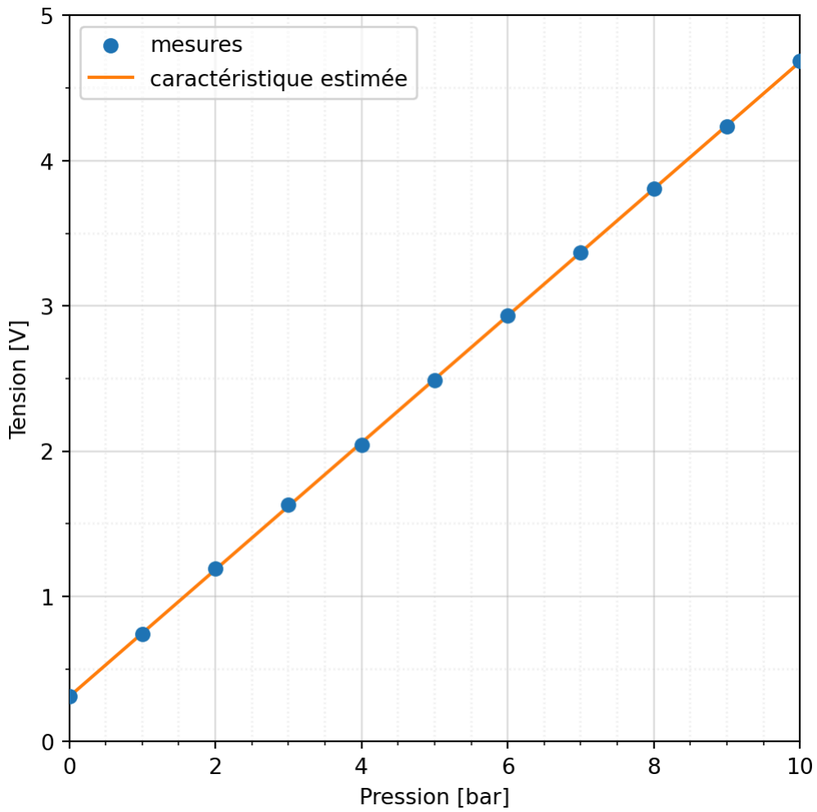
Par défaut, matplotlib choisit automatiquement l'espacement des graduations. Pour un contrôle explicite (division principale, sous-division, format des labels), on utilise le sous-module `matplotlib.ticker` :

```
from matplotlib.ticker import MultipleLocator, AutoMinorLocator

# --- graduations principales (division) ---
ax.xaxis.set_major_locator(MultipleLocator(2))   # 1 graduation tous les 2 bar
ax.yaxis.set_major_locator(MultipleLocator(1))   # 1 graduation tous les 1 V

# --- graduations secondaires (sous-division) ---
ax.xaxis.set_minor_locator(AutoMinorLocator(4))  # 4 sous-divisions entre 2 graduations
ax.yaxis.set_minor_locator(AutoMinorLocator(2))  # 2 sous-divisions entre 2 graduations

# --- grille sur les deux niveaux ---
ax.grid(which="major", alpha=0.4)
ax.grid(which="minor", alpha=0.15, linestyle=":")
```



Élément	Rôle
`ax.set_xlim` / `ax.set_ylim`	fixe les bornes affichées de l'axe
`MultipleLocator(pas)`	place une graduation **principale** tous les `pas` unités
`AutoMinorLocator(n)`	place `n` graduations **secondaires** entre deux graduations principales
`ax.grid(which="major"/"minor")`	affiche la grille correspondant au niveau de graduation choisi

Labels de graduation personnalisés

Pour remplacer les nombres par des labels personnalisés (ex. unités, catégories) :

```
ax.set_xticks([0, 2.5, 5, 7.5, 10])
ax.set_xticklabels(["0", "2.5", "5", "7.5", "10 bar"])
```

```
[Text(0.0, 0, '0'),
 Text(2.5, 0, '2.5'),
 Text(5.0, 0, '5'),
 Text(7.5, 0, '7.5'),
 Text(10.0, 0, '10 bar')]
```

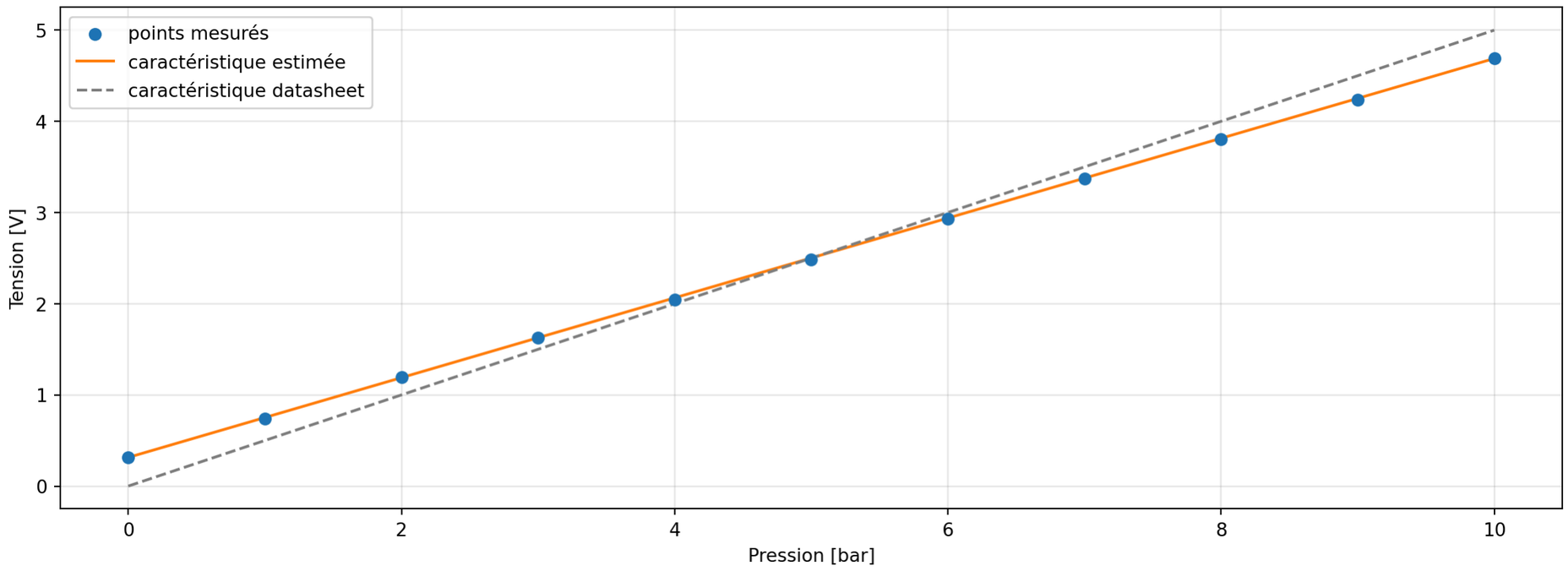
Tip

l'AI est très forte pour aider à trouver la bonne fonction. Il faut juste avoir l'idée que c'est faisable…

2.6.7 Exemple complet : caractéristique estimée vs nuage de points

On combine ici `np.linspace` (pour générer un axe **continu**, indépendant des points de mesure) avec la transformation affine, afin de tracer la **caractéristique théorique** du capteur par-dessus le **nuage de points** réel :

▶ Code



Pourquoi ne pas relier directement les points mesurés ?

`X_ref` ne contient que 11 valeurs : les relier avec `ax.plot(X_ref, V_etal)` donnerait une ligne brisée en 11 segments. Il est intéressant de montrer avec les points non reliés que les valeurs sont peu nombreuses.

Instrumentation 2026-2027

2.7 Exercice 1 : correction d'une caractéristique affine

2.7.1 Contexte

Un capteur de pression (0–10 bar) délivre une tension liée à la grandeur physique mesurée par une relation **affine** :

$$V_{mesure} = a \cdot X + b$$

où X est la pression en bar, a le gain (V/bar) et b l'offset (V). Le constructeur annonce une *sensibilité* $a = 0.500$ V/bar et un offset $b = 0.000$ V, mais le capteur réellement monté sur le banc a dérivé (tolérance de fabrication, vieillissement) : ses vrais a et b sont **inconnus** et doivent être retrouvés par étalonnage.

2.7.2 Fichiers fournis

- `data/etalonnage_capteur.csv` — colonnes `X_ref_bar`, `V_mesure_V` : 11 points obtenus en imposant une pression de référence connue X_{ref} (banc d'étalonnage) et en relevant la tension du capteur.
- `data/acquisition_capteur.csv` — colonnes `t_s`, `V_mesure_V` : signal temporel acquis lors d'un essai (montée puis descente de pression), à corriger.

2.7.3 Travail demandé

1. Charger `etalonnage_capteur.csv` et tracer V_{mesure} en fonction de X_{ref} (nuage de points).
2. Identifier la droite $V = a \cdot X + b$ qui décrit ces points. Cherchez un moyen de calculer $\hat{a}$ et $\hat{b}$. Superposer la droite obtenue au nuage de points.
3. En déduire la relation de correction **inverse**, qui permet de retrouver la grandeur physique à partir de la tension mesurée :

$$\hat{X} = \frac{V_{mesure} - \hat{b}}{\hat{a}}$$

4. Charger `acquisition_capteur.csv`, appliquer la correction du point 3 pour obtenir $X(t)$ en bar.
5. Tracer sur une **même figure à deux sous-graphiques** (`plt.subplots(2, 1, sharex=True)`) : en haut le signal brut $V_{mesure}(t)$ en volts, en bas le signal corrigé $X(t)$ en bar. Titre, labels et unités obligatoires sur les deux sous-graphiques.
6. Comparer $\hat{a}$, $\hat{b}$ trouvés aux valeurs du datasheet ($a_{datasheet} = 0.5, b_{datasheet} = 0$). **Expliquer l'écart observé** : d'où peut-il provenir ? Et surtout : **peut-on faire mieux** que la méthode des deux points extrêmes pour déterminer a et b ? (Indice : que se passerait-il si l'un des deux points extrêmes de l'étalonnage était entaché d'une erreur de mesure inhabituellement grande ?)

2.7.4 Squelette de script à compléter

```python
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

try:
    script_dir = Path(__file__).resolve().parent
except NameError:
    script_dir = Path.cwd()

# --- 1. Étalonnage ------------------------------------------------
X_ref, V_etal = np.loadtxt(
    script_dir / "data/etalonnage_capteur.csv", delimiter=",", skiprows=1, unpack=True
)

# TODO 1 : tracer V_etal en fonction de X_ref (nuage de points)

# --- 2. Identification de a et b (méthode des 2 points extrêmes) ---
# TODO 2 : Trouvez un moyen d'identifier les paramètres de la droite
```

2.8 Sonde NTC : théorie et chaîne de traitement

2.8.1 La thermistance NTC

Une thermistance **NTC** (*Negative Temperature Coefficient*) est une résistance dont la valeur **diminue** quand la température **augmente**. C'est le comportement inverse d'une résistance métallique classique. Sa caractéristique n'est **pas linéaire** : on ne peut donc plus se contenter d'une correction affine.

Le modèle le plus couramment utilisé est la **loi Beta** (approximation de l'équation de Steinhart–Hart) :

$$R(T) = R_0 \cdot \exp\left[B\left(\frac{1}{T} - \frac{1}{T_0}\right)\right]$$

avec :

- T, T_0 en **kelvins** (attention à la conversion $T[K] = T[°C] + 273{,}15$),
- R_0 : résistance de la sonde à la température de référence T_0 (typiquement $T_0 = 25°C$),
- B : coefficient caractéristique de la sonde (en kelvins), donné par le fabricant, typiquement entre 3000 et 4500 K.

Caractéristique de la sonde utilisée dans ce TP :

Paramètre	Valeur
R_0 (résistance à 25°C)	10 000 Ω
T_0	25°C (298,15 K)
B	3950 K

2.8.2 Le montage en pont diviseur de tension

La sonde NTC n'est pas branchée seule : on ne sait mesurer que des **tensions**, pas directement des résistances. On place donc la NTC dans un **pont diviseur de tension**, avec une résistance connue R_{fixe} de même ordre de grandeur que R_0 (pour obtenir la sensibilité maximale autour du point de fonctionnement) :

$$V_{out} = V_{cc} \cdot \frac{R_{ntc}}{R_{fixe} + R_{ntc}}$$

Paramètres du montage :

Paramètre	Valeur
V_{cc}	5 V
R_{fixe}	10 000 Ω

2.8.3 Montage électrique

La figure ci-dessous résume, indépendamment du détail électrique du montage, l'**évolution de la grandeur d'intérêt** au fil de la chaîne de mesure et de traitement, de la température réelle de l'eau jusqu'au résultat :

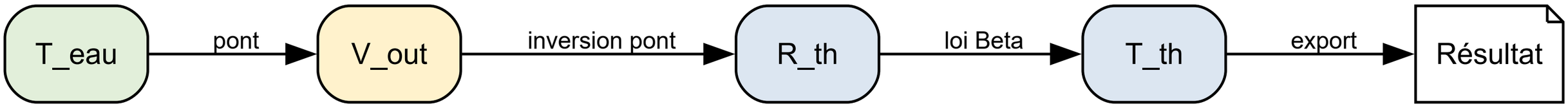


Figure 1: Chaîne de traitement : T_eau (réelle, inconnue) → V_out (mesurée, présente dans le CSV) → R_th et T_th (calculées par le script) → résultat exporté.

Ce qui est donné, ce qui est calculé

- V_{out} est la **seule grandeur mesurée**, présente telle quelle dans le fichier `sonde_ntc_bouilloire.csv` fourni (colonne `V_pont_V`).
- R_{th} et T_{th} **ne figurent pas** dans le fichier fourni : ce sont des grandeurs **calculées par votre script** à partir de V_{out}.

2.8.4 De la tension mesurée à la température : démarche de correction inverse

La correction ne se fait pas en une étape, contrairement au cas affine : il faut **remonter la chaîne de mesure maillon par maillon**.

Étape 1 : de V_{out} à R_{ntc}. On inverse la formule du pont diviseur :

$$V_{out}(R_{fixe} + R_{ntc}) = V_{cc}\, R_{ntc} \quad \Longrightarrow \quad R_{ntc} = \frac{R_{fixe}\, V_{out}}{V_{cc} - V_{out}}$$

Étape 2 :de R_{ntc} à T. On inverse la loi Beta :

$$\ln\left(\frac{R_{ntc}}{R_0}\right) = B\left(\frac{1}{T} - \frac{1}{T_0}\right) \quad \Longrightarrow \quad \frac{1}{T} = \frac{1}{T_0} + \frac{1}{B}\ln\left(\frac{R_{ntc}}{R_0}\right)$$

$$T[K] = \left[\frac{1}{T_0} + \frac{1}{B}\ln\left(\frac{R_{ntc}}{R_0}\right)\right]^{-1} \quad \Longrightarrow \quad T[°C] = T[K] - 273{,}15$$

2.9 Exercice 2 : sonde NTC dans une bouilloire

2.9.1 Contexte

On souhaite suivre la température de l'eau dans une bouilloire à l'aide de la sonde NTC décrite en partie 5, montée dans le pont diviseur de tension (cf. Figure 1). Un système d'acquisition relève la tension $V_{out}(t)$ toutes les 5 secondes, avec un bruit électronique d'écart-type 10 mV.

Le profil attendu est caractéristique d'une bouilloire : l'eau, initialement à 10°C, chauffe jusqu'à ce que le thermostat coupe l'alimentation à 90°C, puis refroidit lentement jusqu'à température ambiante.

2.9.2 Fichier fourni

`data/sonde_ntc_bouilloire.csv` — colonnes `t_s`, `V_pont_V` : tension du pont mesurée au cours du temps (~600 points sur 50 minutes).

2.9.3 Aperçu de la donnée fournie

Comme illustré par la Figure 1, la seule grandeur présente dans `sonde_ntc_bouilloire.csv` est $V_{out}(t)$ — la voici telle quelle, **avant tout traitement** :

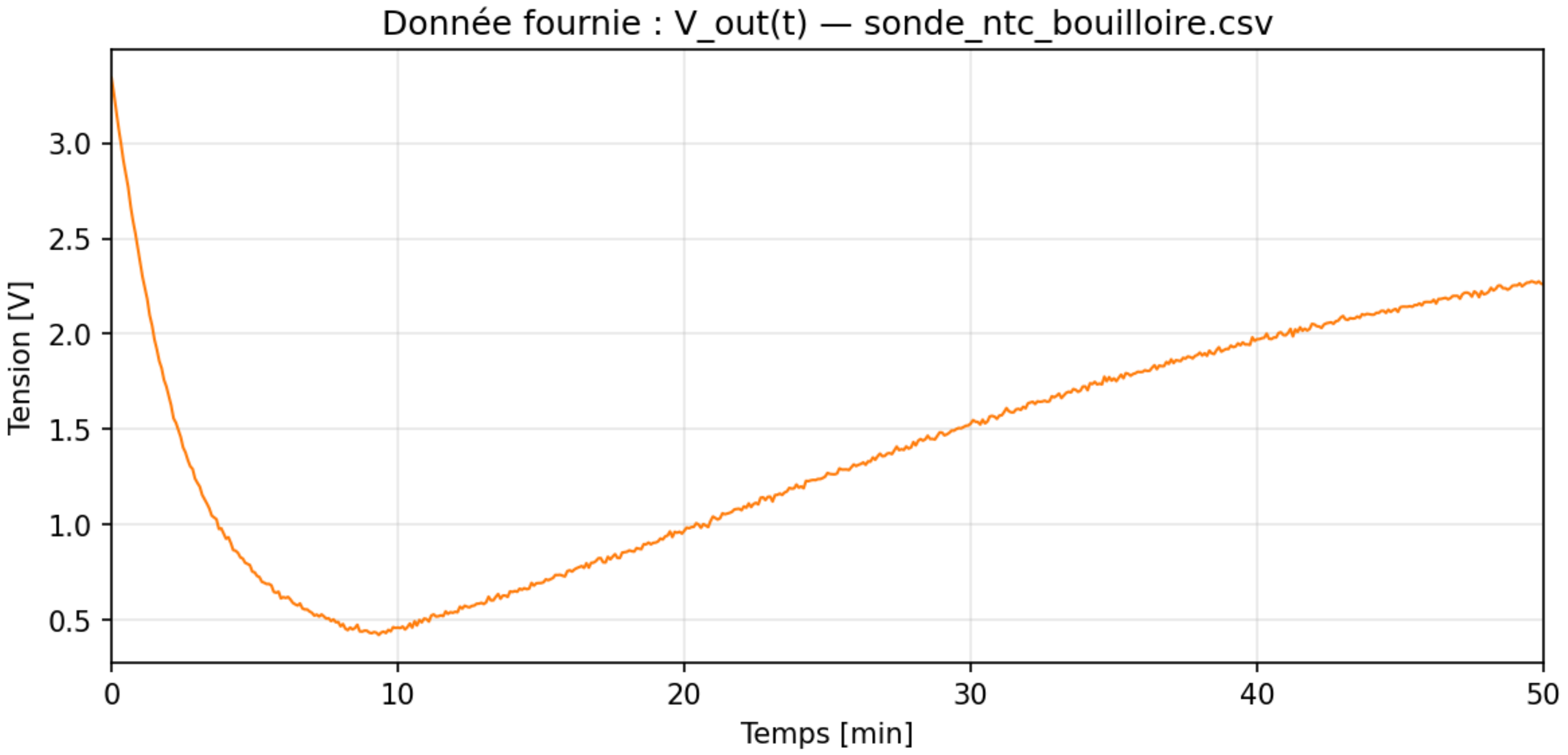


Figure 2: Tension brute V_out(t) telle qu'elle apparaît dans le fichier `sonde_ntc_bouilloire.csv` fourni. C'est la seule courbe que vous pouvez tracer sans calcul : la conversion en température (partie suivante) est à votre charge.

Notez que V_{out} diminue quand T augmente (cohérent avec une NTC placée au dénominateur du pont) : ce n'est **pas encore** la courbe de température recherchée.

2.9.4 Travail demandé

1. Charger le fichier et tracer $V_{out}(t)$ en volts, avec titre, labels, unités et grille.
2. Écrire une fonction `tension_vers_resistance(Vout, Vcc, Rfixe)` qui applique la formule d'inversion du pont (étape 1 de la partie 5).
3. Écrire une fonction `resistance_vers_temperature(R, R0, T0_K, B)` qui applique la formule d'inversion de la loi Beta (étape 2), et renvoie une température en **°C**.
4. Combiner les deux fonctions pour obtenir $T(t)$ à partir de `data/sonde_ntc_bouilloire.csv`, avec les paramètres $R_0 = 10\,000\,\Omega, T_0 = 25°C, B = 3950\,\text{K}, V_{cc} = 5\,\text{V}, R_{fixe} = 10\,000\,\Omega$.
5. Tracer $T(t)$ en °C (temps en minutes en abscisse). Identifier graphiquement (ou par calcul) :
   - l'instant où le thermostat coupe le chauffage (maximum de température) ;
   - la valeur de ce maximum (doit être proche de 90°C) ;
   - la constante de temps approximative du refroidissement.
6. **Question d'analyse** : pourquoi la relation entre V_{out} et T ne peut-elle pas être approximée par une simple droite sur toute la plage 10–90°C ?

2.9.5 Squelette de script à compléter

```python
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

try:
    script_dir = Path(__file__).resolve().parent
except NameError:
    script_dir = Path.cwd()

# --- Paramètres du montage et de la sonde (donnés dans l'énoncé) ---
Vcc     = 5.0         # V
R_fixe  = 10_000.0    # ohm
R0      = 10_000.0    # ohm, résistance NTC à T0
T0_C    = 25.0         # °C
T0_K    = T0_C + 273.15
B       = 3950.0      # K

def tension_vers_resistance(Vout, Vcc, Rfixe):
```

Piste de correction

Le thermostat doit couper aux alentours de $t \approx 550\text{–}560$ s (≈ 9 min), pour une température proche de 90°C, cohérent avec le seuil de coupure d'une bouilloire réelle.

Echange de données

- Cyberlearn

  La section python propose les données des capteurs.
- À déposer sur le Padlet du cours

  Déposez votre graphique de l’évolution de la température sur le padlet du cours.